IndexBudget

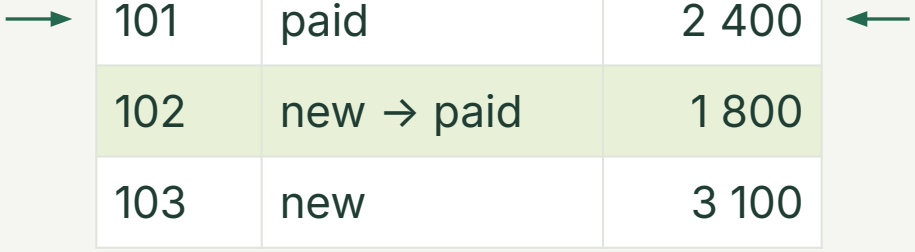


Подбор индексов PostgreSQL с учётом нагрузки и ограничений ресурсов

Нагрузка меняется – какие индексы выбрать?

```
SELECT * FROM orders
WHERE status = 'new';
```

Индекс ускоряет поиск



orders

id	status	total
101	paid	2 400
102	new → paid	1 800
103	new	3 100

```
UPDATE orders
SET status = 'paid'
WHERE id = 102;
```

Замедляет обновление

Какую задачу решают системы рекомендаций

Критерий	Dexter	pganalyze	IndexBudget
Автоматически предлагает индексы	+	+	+
Позволяет ограничить число индексов	–	+	+
Позволяет ограничить оценочные расходы на запись	–	+	+
Позволяет задать бюджет места для индексов	–	–	+

IndexBudget: один набор индексов для нескольких режимов нагрузки — в рамках заданных бюджетов.



Цель: автоматизировать подбор индексов, учитывая разные сценарии нагрузки

1. Учитывать несколько сценариев нагрузки.
2. Подбирать **единый набор индексов** по худшему сценарию.
3. Соблюдать лимиты на объём, запись и число индексов.
4. Показывать оценку эффекта и объяснять выбор.

Проверяем комбинации до создания индексов



Анализ запросов и схемы

1. Из pg_stat_statements

Запросы и их частоты получаем из статистики PostgreSQL

Что умеем разбирать

SELECT: колонки, *, алиасы, INNER JOIN, ORDER BY, LIMIT.
Фильтры: сравнения, AND / OR, IN, BETWEEN, IS NULL / IS NOT NULL.
UPDATE / DELETE: одна таблица с WHERE; в SET — константы.

2. Из JSON

Пользователь сам пишет SQL запросы и их частоты

Что пока не поддерживаем

CTE, подзапросы, UNION
GROUP BY, агрегаты, оконные функции и вычисляемые выражения
OUTER / CROSS JOIN, INSERT и MERGE

Формирование списка индексов

Как получаем список
Из фильтров, JOIN и сортировки

```
SELECT id, payload FROM records
WHERE group_id = 7 AND value >= 900
ORDER BY value LIMIT 50
```


B-tree: (group_id, value)
Вариант: INCLUDE (id, payload)

Другие индексы
Hash — для равенства
BRIN — для диапазонов

Отбор по ограничения

Пользователь задает ограничения

- Память
- Нагрузка
- Количество

Оценка расходов на запись

Изменения строк/ч × (ключи + INCLUDE)

Как выбираем набор

1. Перебираем кандидатов.
2. Выбираем лучшего.
3. Повторяем добавления, затем проверяем замены.

Оценка стоимости запросов

1. HypoPG моделирует выбранные индексы
2. EXPLAIN строит план запросов (без выполнения)
3. Сохраняем Total Cost каждого плана.



Сравнение при разных сценариях

Как считаем сценарий

- Умножаем cost каждого SQL на частоту и складываем.
- Сравниваем с тем же сценарием без новых индексов.

- mean — улучшить средний результат.
- robust — максимизировать минимальное улучшение по сценариям

Критерий	Среднее M	Худшее R
mean	54,6%	83,2%
robust	58,3%	68,8%

Итоговые рекомендации

предложенные индексы

Таблица	Индекс
groups	btree (category, id) INCLUDE (name)
records	btree (value) INCLUDE (id, payload) btree (group_id) INCLUDE (id, value)

влияние на запросы

SQL	До	С набором	Изменение
SQL 1 SELECT	6 713,6	7,5	Снижение на 99,9%
SQL 2 SELECT	6 713,7	27,6	Снижение на 99,6%
SQL 3 SELECT	6 720,1	3 352,1	Снижение на 50,1%

Команда



Backend dev на GLM
метрики, бюджеты и проверочные эксперименты



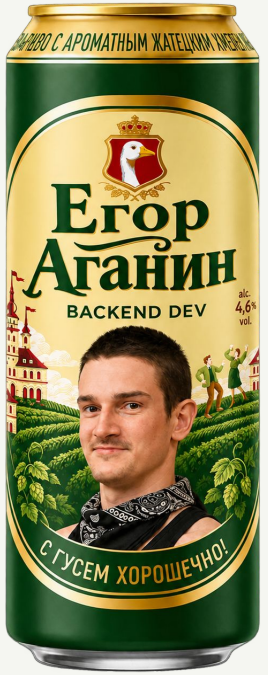
Backend dev на Codex
анализ SQL и подготовка нагрузки



Backend dev на Codex
управление командой, CLI, orchestration и поиск



Backend dev на Codex
генерация кандидатов и HTML-представление



Backend dev на Codex
PostgreSQL и HypoPG

Спасибо!



Теперь покажем, как это работает